main

Linn Erle Kloefta

April 2026

1 Introduction

1.1 Background and Motivation

Handwritten digit classification using the MNIST dataset is a classic benchmark problem in computer vision and machine learning. While modern deep learning frameworks (such as PyTorch or TensorFlow) provide high-level abstractions to build models effortlessly, this project is motivated by a desire to understand the fundamental mathematical operations behind neural networks. By implementing the architecture entirely from scratch using foundational tools like NumPy, we obtain a deeper, transparent view into how models actually learn.

1.2 Problem Definition

The primary objective of this project is to build functioning neural networks capable of correctly classifying 28×28 grayscale images of handwritten digits (0-9) from the MNIST dataset. The challenge is characterized by strict constraints: we are not permitted to use any pre-built deep learning libraries. Instead, we must manually design, implement, and orchestrate the matrix multiplications, activation functions, loss calculations, and gradient derivations forming the backpropagation pipeline.

1.3 System Overview

To tackle the classification problem, two distinct neural network architectures are manually implemented and evaluated in this project:

1. **Multilayer Perceptron (MLP):** A standard 2-layer fully connected network incorporating a hidden layer of 128 neurons activated by a Sigmoid function, and a 10-neuron output layer activated by Softmax.

2. **Convolutional Neural Network (CNN):** A 2-layer network extracting spatial hierarchies, comprising a Convolutional Layer (5×5 kernel, ReLU activation) flattened into a Fully Connected Layer (Softmax activation).

2 Methodology

2.1 Algorithms and Architecture

The project requires the complete manual implementation of the forward sequence, performance evaluation (loss), and backward sequence (gradient traversal) for two distinct models.

2.1.1 Multilayer Perceptron (MLP) Implementation

The MLP translates a flattened 784-dimensional image vector through a hidden feature space of 128 neurons before compiling logits for the 10 digit classes. The

forward sequence dictates:

$$Z^{[1]} = XW^{[1]} + B^{[1]}, \quad A^{[1]} = \sigma\left(Z^{[1]}\right) = \frac{1}{1 + e^{-Z^{[1]}}} \tag{1}$$

$$Z^{[2]} = A^{[1]}W^{[2]} + B^{[2]}, \quad A^{[2]} = softmax\left(Z^{[2]}\right) \tag{2}$$

During the backward pass (refer to Figure **??**), gradients are propagated from the cross-entropy cost function using the chain rule. The explicit parameters update follows vanilla Gradient Descent logic, systematically penalizing weights based on their specific contribution to classification error averages across the batch.

2.1.2 Convolutional Neural Network (CNN) Implementation

Unlike the flattened MLP, the CNN natively parses the 2D configuration (28×28) of the image. The forward pass applies a valid-mode 2D convolution sweeping over the input pixels:

- **Convolution Output**: A localized dot-product between a defined $K \times K$ kernel and the target image region, followed immediately by a ReLU activation ($A = \max(0, Z)$).
- **Fully Connected Sequence**: The localized feature maps are then flattened to intersect with a dense weight matrix resolving into Softmax distribution predictions.

2.2 Initialization and Numerical Stability

To jumpstart training and prevent mathematical dead-ends, weights are carefully initialized using scaled random normal distributions (e.g., He or Xavier initializers). Special care is taken inside the Softmax implementation by subtracting maximum row values prior to exponentiation explicitly preventing ‘NaN‘ overflows, and a tiny ϵ is paired alongside logarithmic operations to mitigate division-by-zero bounds during loss evaluations.

3 Evaluation

3.1 Experimental Settings

Training strictly leverages batch-based operations (batch size of 128 items from the PyTorch DataLoader) mapping cleanly into our matrix dot products. The Multi-Layer Perceptron (MLP) was comprehensively evaluated over 30 epochs employing a foundational learning rate (η) of 0.5, while the CNN traversed its localized feature extraction mapping utilizing fine-tuned constraints.

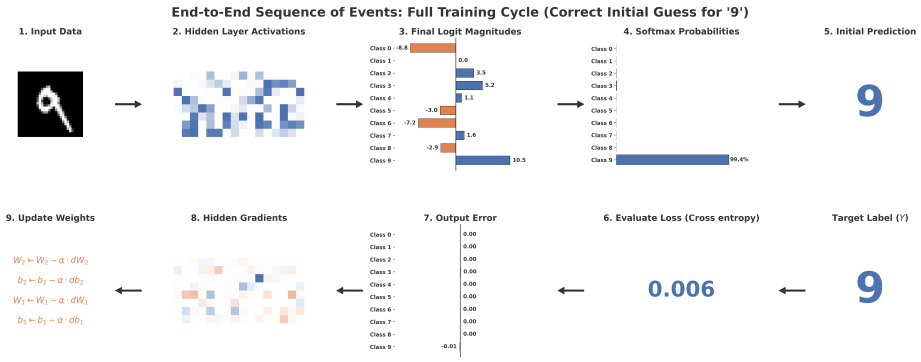

Figure 1: End-to-End sequence of events spanning the Full Training Cycle, visualizing actual array manipulations across the forward pass (top) and the corrective backward pass gradients (bottom).

3.2 Training Progression and Feature Extraction

As observed in our empirical metrics, the model displays robust convergence stability. The Cross-Entropy loss smoothly decays while overall Test Accuracy steadily rises, breaking past 90% accuracy thresholds within limited iterations (Figure **??**).

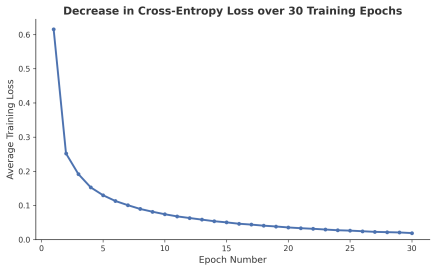

Figure 2: Decrease in Cross-Entropy Loss (MLP)

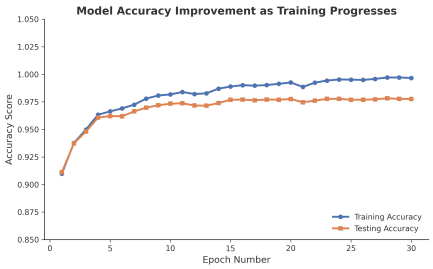

Figure 3: Accuracy trajectory indicating no severe overfitting

Delving deeper into feature extraction, evaluating the 128 hidden neurons provides remarkable intuition regarding the mapping characteristics generated by the model.

3.3 Qualitative Profiling and Confusion Analysis

Sample step-by-step predictions visually validate the model's forward propagation fidelity (Figure **??**).

To definitively document overall performance barriers, the model's total interactions across 10,000 distinct test samples were synthesized within a detailed

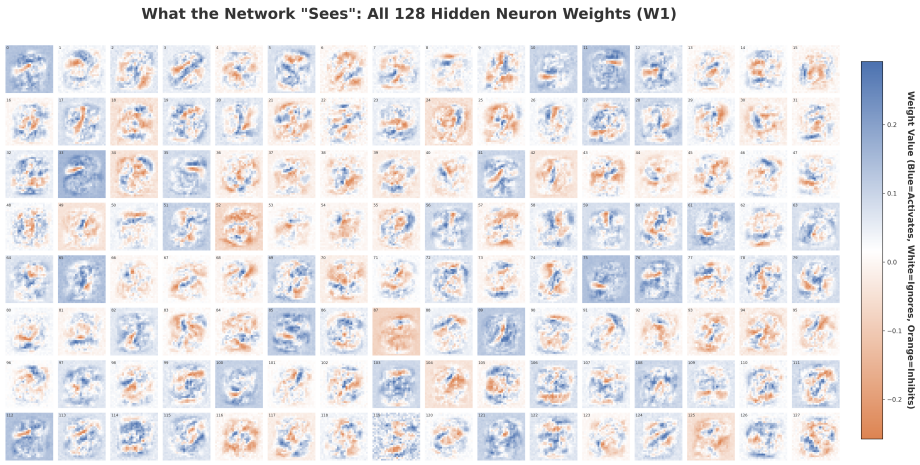

Figure 4: Representation of learned weights across all 128 Hidden Neurons in W1. Organized as a 8×16 lattice directly corresponding to hidden layer activations.

semantic Confusion Matrix (Figure **??**). Additionally, hyperparameter analysis confirms that shifting the quantity of allocated hidden neurons directly correlates with capacity breakpoints scaling network accuracy (Figure **??**).

4 Learning Outcomes

4.1 Tasks Accomplished

Over the course of this initiative, the core logic mapping for robust Neural Networks (both MLP and CNN architectures) was successfully achieved operating strictly utilizing pure NumPy matrix operations. We executed fully bespoke dataloading, unabstracted Forward Propagation layers (utilizing ReLU, Sigmoid and custom stable Softmax bindings), mathematically derived Backward Propagation cost derivatives mapped via careful chain-rule calculus, and effectively evaluated the resulting network implementations via detailed diagnostic visual metrics.

4.2 Challenges and Debugging Realities

Several profound constraints were encountered navigating mathematical thresholds organically. Chief among them involved floating point limits—specifically ‘NaN‘ returns spawned rapidly from exponential algorithms executing excessively large gradients (Gradient Explosion) and Cross-Entropy loss functions returning ‘-inf‘ errors evaluating arrays matching exactly zero. Effectively navigating these barriers heavily required establishing internal offsets (like epsilon

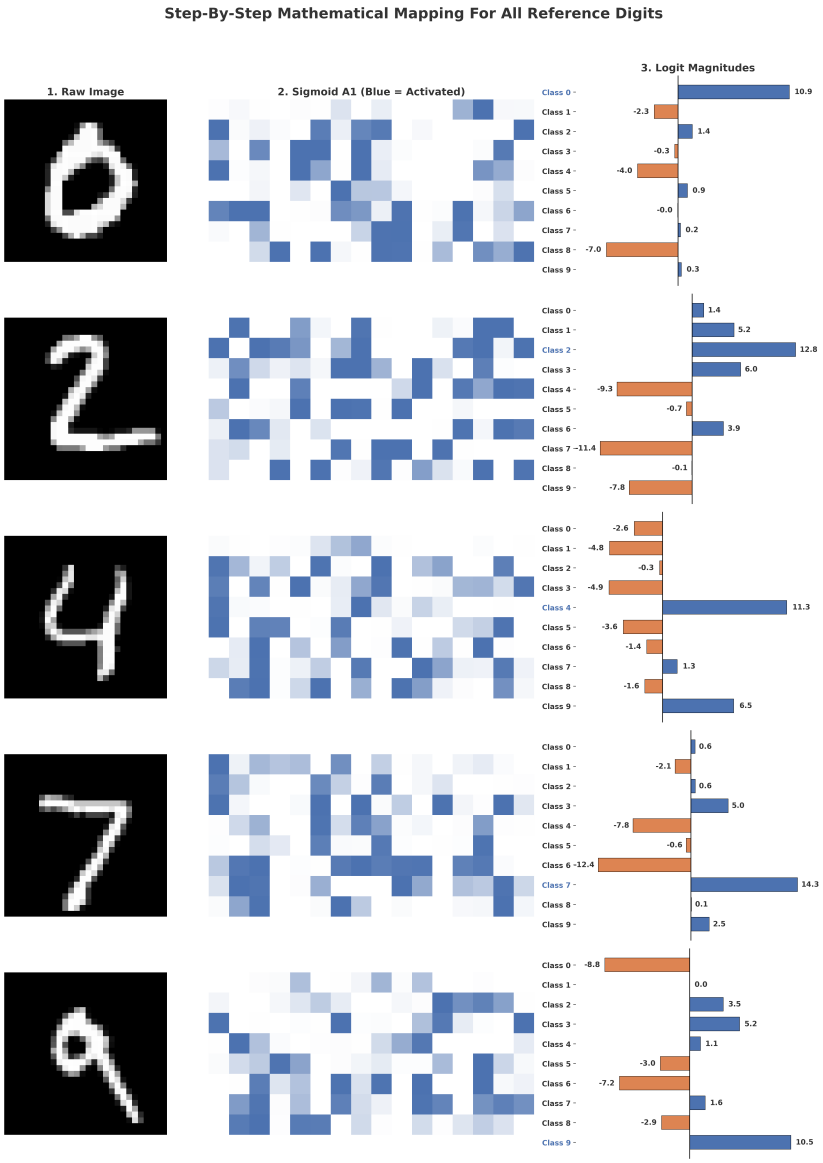

Figure 5: Manual inspection illustrating exact network representations (Input, Hidden map, and Logits) for a subset of global reference digits.

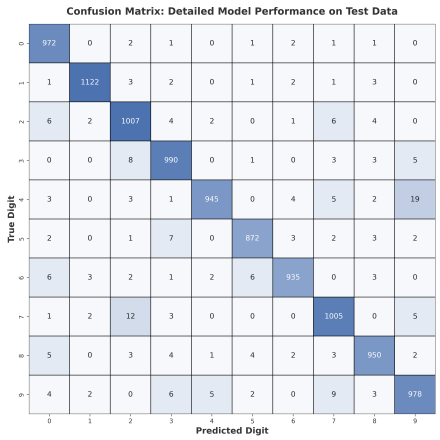

Figure 6: Evaluation of Test Predictions (Confusion Matrix)

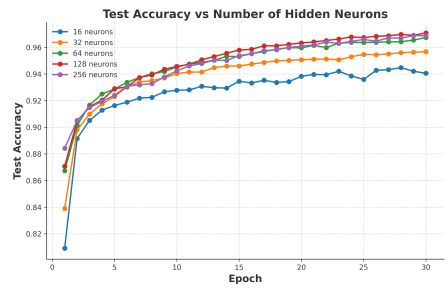

Figure 7: Impact of Hidden Neuron Counts on Test Accuracy

values inside logarithms) and numerically stable maximums before Softmax distributions scaled, granting genuine stability directly into the backpropagation cycle.

4.3 Lessons Learned

The manual reconstruction of algorithms typically handled safely behind high-level APIs (like PyTorch) significantly clarified why granular hyperparameter initializations govern model feasibility. Observing empirical evidence mapping localized structural weight limits confirmed exactly how gradients flow iteratively across dense architectures, delivering foundational insight paramount for advanced machine learning workflows hereafter.